Project Verhalenbank Report - Multilingual Question Answering

Group 1: Marten de Jong (S4557182), Jasper Kleine (S5152372) & Fleur van Ast (S4109287)

Semantic Web Technology LIX002M05
Semester 1a, Master Information Science
F. Pannach, G. Bouma
28.10.2026

Table of Content

Motivation and Research Question 2
Relevant Scientific Work 2
Method 4
Data 4
Python Scripts & Notebook 4
Google Sheets 4
LLM experiments QA - Jupyter Notebook 6
Prompt Iteration 7
Models 7
Evaluation Metrics 7
Results 8
Overall performance 8
Performance Across Prompting Strategies 8
Cross-Lingual Evaluation 9
Table 5 9
Cross-lingual performance for GPT-5.6-luna 9
Discussion 10
Error Analysis 10
1. Ontological Abstraction 10
2. Cross-Lingual Entity Leakage 11
3. Diacritic Inconsistencies & String Matching in Frisian 11
References 11
Appendices 12

Motivation and Research Question

The “Volksverhalenbank van de Lage Landen” (Koman, 2006) includes around 100.000 stories in various genres and from different time periods. Stories are provided in Dutch, Frisian and a variety of regional languages. However, for this project, we decided to select only Dutch stories for our ground truth data, for the purpose of testing the Question Answering (QA) performance of Large Language Models (LLMs) in multiple languages, including Frisian. It is unclear whether the language of a question changes how well a model answers about content that exists in another language (Dutch in our case). Fictional and folktale stories are most likely harder to interpret for an LLM instead of real-world knowledge and facts. It would be interesting to see how well an LLM can answer various questions about these kinds of stories and if certain languages influence these results, as ambiguity may arise and this can cause difficulties in both providing a suitable answer and possibly its translation.

This leads us to the following research question: “What effect does the language of the question (Dutch, English, German or Frisian) have on the accuracy of an LLM when answering questions about Dutch stories from the Verhalenbank?” Other interesting research directions that we want to address are if providing the ontology and source texts improves the models performance and if we can see an effect on a low-resource language, namely Frisian.

It is expected that the model's accuracy will be the highest for the English questions, unless the source text is provided with the question. After all, source texts are provided in Dutch from the Verhalenbank. Translation errors or ambiguity will influence the results. A low-resource language like Frisian is expected to yield lower accuracy. Results will likely vary across question types, we expect that a question such as: “What is the location of the story” is easier to answer for an LLM, through Named Entity Recognition (NER), than for example a question about a relationship or situation that has to be inferred.

To examine these research questions and hypotheses, we tried different experiments on two LLM’s, which included questions in four languages (German, English, Frisian) with the baseline being Dutch. That is, because the source texts are in Dutch. To reliably evaluate and compare the performance of the LLM across languages, we manually curated ground truth triples from randomly sourced texts from the Verhalenbank, by topic. Our Ontology was then transferred to WebProtégé.

Relevant Scientific Work

It is known that models such as GPT are doing well on a variety of language tasks when it is provided in English (Qin et al., 2023; Zhang et al., 2023). This results from the fact that most training data is in English (Brown et al., 2020). Earlier research about the multilingual abilities of LLM’s include Zhang et al. (2023), who studied how LLMs acquire multilingual abilities and their performance variation across languages, among them reasoning and knowledge access. For this, they used a novel prompting method: response back-translation (RBT), and prompt translation (PT). Their findings show that the GPT3.5 achieves higher performance on tasks that can be translated without altering the correct output and that multilingualism can limit performance even in language independent tasks.

Shen et al. (2023) introduced a benchmark for bilingual, Chinese-English, role evaluation for LLMs (RoleEval), which is necessary for establishing real world knowledge, and providing more immersive interactions. They use multiple-choice questions about people and fictional characters drawn from multiple domains such as comics, fictions and TV series. Experiments include zero- and few shot settings about, among others, timeline reasoning, event participant reasoning and character relationship reasoning. Their findings show that GPT-4 outperforms other models in its ability to do so. However, in 2024, Yu et al. presented their Literary Fiction Evaluation Dataset, aiming to evaluate fiction comprehension and reasoning of LLMs in Chinese. Results showed that the models struggled considerably regarding answering questions about literary fiction. In the zero- shot setting, GPT reached only 57.08% accuracy, at its highest performance. They state that even prompt- and novel length will have an impact on the results, varying on the specific task.

There is an ongoing discussion about the capabilities of LLM in comparison to Knowledge Graphs (KGs), KG's utility and value nowadays and their ability to even construct these. Petroni et al. (2019) studied BERT and ELMo on their ability to store relational knowledge (entities, common sense and QA) and found that BERT-large performs very well, even competitive with non-neural supervised alternatives. An important distinction is that Petroni et al. (2019) studied if this information was present in the weights of the pre-trained model rather than its ability to retrieve the information from a source text. Additionally, Omar et al. (2023) used a comparative framework to examine the performance of chat-GPT and chatbots for knowledge graphs (KG's) in Question Answering (QA) in English, in particular, their differences. Results show that the optimal model would include KG knowledge infused with GPT-abilities such as question understanding and robustness.

Method

To evaluate LLMs answers to multilingual questions about Dutch stories from the Verhalenbank, and how this depends on the language of the questions, the following setup was decided on. Questions and Ontology (Google Sheets) were hand-curated. Each question was available in English, German, Frisian and Dutch and had a gold answer (subject, object, predicate triple) in an RDF ontology. Every question was introduced to the model separately and under six conditions that varied from zero- shot to multi-shot approaches to control how much information the model was presented with. Availability of story text, ontology (full schema - with instances), ontology (relations) and distractor story (+answer) were controlled factors in this. Models were queried through batch APIs with a fixed instruction in English and were told to output a JSON triple.The answers were evaluated per field and we kept track if the label was in the requested language.

Data

Data used for this project were Dutch stories retrieved randomly from the Verhalen Bank JSON dataset.

Python Script & Notebook

Script “retrieve_files”: After we decided on our final approach, which includes formal triples including class hierarchy and domain/range specifications, we decided to build a python script to randomly select Dutch stories for us. The script automatically selects and retrieves the stories’ metadata from the JSON Verhalenbank file, based on each member's two chosen keywords (Marten: spoken and water, Jasper: draak and toverij, Fleur: kat and hond).

Jupyter Notebook - Running the multilingual Question Experiments: The notebook reads the QA dataset, story metadata and the full ontology (from webProtégé), then builds a request for every question under six prompt conditions (zero-, one- and few-shot; see LLM experiments section). The requests are submitted as test and full batches to the OpenAI or Gemini batch APIs. The model’s JSON answers (subject, predicate, object) are then retrieved and scored against the expected triples.

Google Sheets

Tab 1: Ontology Schema, this schema contains the Predicates, Domains and Ranges we used for our Ontology triples. Each row contains an explanation of such. We used these for the QA datasheet for the ground truth. Please refer to table 1 in Appendix A for the full Ontology schema.

Tab 2: Stories + Metadata
Stories were sourced randomly (see Python script: retrieve_files.py) from the JSON dataset Verhalenbank. In this tab, the selected stories’ full texts, identifiers and additional data (as described underneath) are gathered.

The selected metadata information was displayed in the following rows: "story_id", "identifier" , "Annotator", "chosen_keyword", "keyword_prevalence", "original_language", "region", "province", "subgenre", "subject_atu_sinsag", "motif", "named_entities" "named_locations", "word_count", "description", "main_text", "all_tags".

Tab 3: QA dataset
The content of this tab covers the questions in our four languages: baseline (Dutch) and translations (English, German, Frisian) and their answers in the form of Ground Truth Triples. Questions and their valid answers, including all translations, were created manually.

LLM experiments QA - Jupyter Notebook

System Prompt (SP): Ontology schema only (without instance data)
Output requirement:

> “Return exactly one valid JSON object with exactly these keys:
> {"subject": "...", "predicate": "...", "object": "..."}
>
> Answer the question using only the supplied context. Use "unknown" when a requested value cannot be determined.
>
> The predicate must be the exact ontology property identifier, never its translated or human-readable label. For example, output "appearsIn", not "komt voor in verhaal".
>
> Preserve canonical identifiers exactly, including capitalization, punctuation, and underscores. For example, output "Item_47370", not "Item 47370".
>
> Natural-language subject and object labels may use the language of the question, but canonical story and entity identifiers must not be translated or reformatted.
>
> Before returning the answer, verify that all three keys are present, the JSON is valid, the predicate is an ontology identifier, and identifiers are unchanged. Output JSON only, with no Markdown or explanation.”

User Prompt terminology:
The **Ontology** refers to the complete filled ontology graph (stories.rdf). **Story** means the main text of a story.
Question refers to the question itself, See tab 3, QA_Dataset tab in the Google Sheets for the full list of curated questions. For example: NL: “Waar komt de zeer vrome vrouw vandaan?”, EN: Where does the very pious woman come from?, DE: “Wo kommt die sehr fromme Frau her?” and in FY: “wêr komt de tigefromme frou wei?”

For each language (Frisian, English, German and Dutch as baseline), the following experiments were conducted:

1. Zero-shot: SP + Question
2. Zero-shot - Ontology: SP + Ontology + Question
3. One-shot: SP + story + question
4. One-shot - Ontology: SP + ontology + story + question
5. Few-shot: SP + correct story + 1 wrong story + question
6. Few-shot - Ontology: SP + ontology + correct story + 1 wrong story + question

We chose these experiment conditions to examine models’ performance under different prompting conditions and with different information at hand. The first experiment entails the least information with only the question and the ontology relations provided. Each condition contains one variant where the full ontology is provided and one without. Later experiments (5&6) include a distractor story (wrong story) to test consistency.

Prompt Iteration

After the first results we changed the prompt as results were not as good as expected. The following prompt is what we continued with in later experiments:

> “Return exactly one valid JSON object with exactly these keys:
> {"subject": "...", "predicate": "...", "object": "..."}
>
> Rules:
> 1. Extract the concrete character, entity, location, or value directly from the story or RDF facts. NEVER output abstract ontology class names or URIs (such as "Character", "Location", or "http://...") as subject or object.
> 2. The predicate must be the exact ontology property identifier (e.g., "appearsIn", "livesIn", "usesObject"). Never use labels, natural descriptions, or full URIs.
> 3. Subject and object values must match the language of the question (translate the extracted Dutch terms into the question's language). Canonical identifiers (e.g., "Item_47370") must stay exactly as they are.
> 4. Only use "unknown" if the context truly contains no information to answer the question.
> 5. Return JSON only, with no Markdown ticks or commentary.“

One significant change is the fact that we ask the model that it must match the language of the question and never use any abstract ontology class names or URIs. Gemini specifically was susceptible to falling back to these abstract answers.

Models

The models we used for the experiments were OpenAI GPT-5.6-luna and Google Gemini-3.5-flash-lite. Setting: reasoning effort (low). The main argument for choosing these two API-based models was time and these models having the capability of handling West-Germanic languages. As these models are API-based, running models locally is not necessary, as this can take up too much time and taking an internal look into certain kinds of architectures was not our goal. Furthermore, as these models are large models, we suspect these have at least some cross-lingual capabilities, instead of smaller models that may only be trained on a single language. Another thing is that the API-based approach could also fit into our approach of providing data and managing the output. Working in batches enabled us to try different approaches to these models.

Evaluation Metrics

We used two metrics, one more strict and one more loose. First, any Language Accuracy: a triple <s,p,o> is correct if the predicate matches the canonical ontology property and both subject and object correspond to valid reference entities in any supported language.
Secondly, requested language accuracy: requires the subject and object to strictly match the gold standard annotations in the specific language of the prompt.

Results

We evaluated the performance of GPT-5.6-luna and Gemini-3.5-flash-lite on 1440 batch requests.

Overall performance

As summarized in table 2, GPT-5.6-luna achieved the highest overall Any Language Accuracy (52.8%), outperforming the base Gemini-3.5-flash-lite model (45.4%). Implementing prompt constraints in Gemini V2.1 substantially bridged this gap, reaching a score of 50.3%.

Table 2

Overall and slot-level extraction accuracy across models

Metric / Dimension	GPT-5.6-luna	Gemini 3.5 (Base)	Gemini 3.5 (V2.1)
Overall Accuracy (Any Language)	52.8% (761)	45.4% (654)	50.3% (724)
Requested Language Accuracy	23.3% (336)	17.9% (258)	24.7% (356)
Triples Accepted via Other Lang.	425	396	368
Predicate Accuracy	84.2% (1,212)	67.4% (970)	75.3% (1,085)
Subject Accuracy (Any Language)	68.3% (984)	53.2% (766)	62.2% (896)
Object Accuracy (Any Language)	65.8% (947)	53.6% (772)	62.0% (893)

Performance Across Prompting Strategies

As can be seen in table 3, zero-shot prompting received a score of zero across all models. Without story text or RDF triples, models correctly generated unknown values rather than hallucinating arbitrary folk tale entities but did not manage to get any correct answer. Incorporating the RDF graph yielded substantial performance improvements in all the strategies it was introduced into. It can also be concluded, as seen in table 3, that adding a distractor text has a negligible effect on performance.

Table 3
Semantic triple accuracy (any language) across prompting and context conditions

Strategy	GPT-5.6-luna Acc.	Gemini Base (V2)	Gemini Rerun (V2.1)
Zero-shot	0.0% (0 / 240)	0.0% (0 / 240)	0.0% (0 / 240)
Zero-shot + Ont.	60.4% (145 / 240)	51.2% (123 / 240)	60.0% (144 / 240)
One-shot	50.4% (121 / 240)	29.2% (70 / 240)	42.9% (103 / 240)
One-shot + Ont.	75.8% (182 / 240)	80.4% (193 / 240)	80.0% (192 / 240)
Few-shot	51.2% (123 / 240)	32.5% (78 / 240)	42.1% (101 / 240)
Few-shot + Ont.	79.2% (190 / 240)	79.2% (190 / 240)	76.7% (184 / 240)

Cross-Lingual Evaluation

Table 4 details the stark difference if the answer has to strictly align with the ground truth. One notable thing is that West Frisian has such a low performance, where 128 cases had to be ‘rescued’ by other languages. In the discussion section this is talked about, most likely having to do with diacritics being present in the answers and questions.

Table 4
Cross-lingual performance for Gemini-3.5-flash-lite (V2 with updated prompt)

Language	Any Lang. Acc.	Req. Lang. Acc.	Rescued by Other
Dutch (NL)	52.2% (188)	49.4% (178)	10
English (EN)	53.3% (192)	21.4% (77)	115
German (DE)	47.5% (171)	15.6% (56)	115
West Frisian (FY)	48.1% (173)	12.5% (45)	128

Table 5
Cross-lingual performance for GPT-5.6-luna

Language	Any Lang. Acc.	Req. Lang. Acc.	Rescued by Other
Dutch (NL)	51.9% (187)	51.4% (185)	2
English (EN)	54.2% (195)	15.8% (57)	138
German (DE)	52.2% (188)	14.4% (52)	136
West Frisian (FY)	53.1% (191)	11.7% (42)	149

Discussion

Our results show that the language of the question has little influence on the ability of the model to find the correct triple for it, with especially high scores for predicate accuracy. Models seem to retrieve the Dutch labels from the source often, even in the conditions where the full RDF was available, indicating that the model fails or refuses to translate. Overall, as expected, accuracy for the Frisian language scores were slightly lower than other languages. Additionally, we find that providing context mattered most: while zero-shot scored 0%, providing the ontology raised accuracy to about 80%. Because the prompts contained the answers as RDF facts, our setup mainly tests question-to-predicate mapping, cross-lingual entity linking and answer verbalization.

Our approach to evaluate the mulitilingual performance of LLMs, had methodological strengths. Firstly, we tested multiple approaches (zero-, one- and few- shot prompts) to distinguish the influence of the performance regarding the availability of the Ontology and source texts. Next, our ground truth triples are defined in a structured way, therefore evaluation and scoring are objective and reproducible. The other approach would have been omitting this structure and evaluating performance of free text generation. Additionally, we compare four languages across questions, including a low-resource language.

However, our results should be interpreted with caution, as our approach has certain limitations. We only used languages from the West-Germanic branch of the Indo-European language family. Therefore our results and conclusions are bound to those and not generalizable. The sample was relatively small and questions and their answers were curated by hand. This is error- and bias prone. The scoring depends strictly on the hand-curated triple. This approach is sensitive to small errors, synonym usage and ambiguity of the question or provided answer. Some questions may have had several valid answers. We had no system to equally distribute question types or their level (easy, medium, hard). Next, using closed source models results in the fact that we can't verify if the Verhalenbank data was in the training data, therefore it is hard to say if the rather poor performance stems from there, or if the models are poor reasoners in the fiction/story domain, both in the language of the source text, as well as the other tested languages. However, we also didn't provide an experiment where we tested performance without the instruction to answer only from context. Another limitation is that we set our reasoning effort to low, to restrict computation time and cost, however limiting the optimal capabilities of the models. Lastly, GPT's performance could have been improved if we had tested the improved prompt on this model as well. Right now, testing conditions are not entirely fair.

Future work could include keeping the Frisian and regional languages source texts as ground truth data to examine if a model is able to translate these and perform QA tasks on them. A wider variety of (open) source models and languages, including a more extensive set of QA sets would improve the quality of this research. Additionally, it would be interesting to implement the use of an evaluation approach that tolerates variations of answers and addresses ambiguity and translation errors. Lastly, an experiment setting where models are tested without providing RDF would test NER and information retrieval.

Error Analysis

1. Ontological Abstraction

When using Gemini without a prompt it would often occur that the model would output ontology class URIs instead of textual answers.

- Question (NL): “waar komt de zeer vrome vrouw vandaan?”
- Target: bale
- Expected: {"subject": "vrome vrouw", "predicate": "livesIn", "object": "bale"}
- Gemini: {"subject": "http://example.org/verhalenbank/Character", "predicate": "livesIn", "object": "http://example.org/verhalenbank/Location"}

To resolve this we updated the prompt with explicitly stating never to output abstract ontology class names or URIs as subject or object.

2. Cross-Lingual Entity Leakage

When answering questions in the other languages based on the dutch source text, models accurately identified the entity but used the dutch form directly rather than translating it to match the query language.

- Question (EN): “How does the story end?”
- Target: flames / burns / fire
- GPT: {"subject": "Item_9577", "predicate": "hasEnding", "object": "vuur"}

This is semantically correct but this causes the Requested Language Accuracy score to drop below 25%.

3. Diacritic Inconsistencies & String Matching in Frisian

Due to encoding during CSV compilation, several non-ASCII diacritics (e.g. û, ê, â) were removed and replaced with whitespace. Item Marten_08_02 (“Yn hooker lyts bist feroaret de tovener himsels…”), the gold standard object for Frisian was stored as “m “ instead of “mûs”. For item Marten_09_02 the same thing occurred, the target label “ûnderwetterlân” was recorded as “ nderwetterl “. When the LLM generated the correct label “mûs” it would consequently not find a match and therefore mark it as an incorrect answer.

References

1. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., (...) & Amodei, D. (2020). Language models are few-shot learners. *Advances in neural information processing systems*, *33*, 1877-1901. proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf?ref=hackernoon.com
2. Koman, R. A. (2006). *Kantjil het dwerghert*. Volksverhalenbank van de Lage Landen. https://verhalenbank.nl
3. Omar, R., Mangukiya, O., Kalnis, P., & Mansour, E. (2023). Chatgpt versus traditional question answering for knowledge graphs: Current status and future directions towards knowledge graph chatbots. *arXiv preprint arXiv:2302.06466*. https://arxiv.org/abs/2302.06466
4. Petroni, F., Rocktäschel, T., Riedel, S., Lewis, P., Bakhtin, A., Wu, Y., & Miller, A. (2019, November). Language models as knowledge bases?. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)* (pp. 2463-2473). Language Models as Knowledge Bases?
5. Qin, C., Zhang, A., Zhang, Z., Chen, J., Yasunaga, M., & Yang, D. (2023, December). Is ChatGPT a general-purpose natural language processing task solver?. In *Proceedings of the 2023 conference on empirical methods in natural language processing* (pp. 1339-1384). 2023.emnlp-main.85.pdf
6. Shen, T., Li, S., Tu, Q., & Xiong, D. (2023). Roleeval: A bilingual role evaluation benchmark for large language models. *arXiv preprint arXiv:2312.16132*. 2312.16132
7. Yu, L., Liu, Q., & Xiong, D. (2024, May). LFED: A literary fiction evaluation dataset for large language models. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)* (pp. 10466-10475). LFED: A Literary Fiction Evaluation Dataset for Large Language Models
8. Zhang, X., Li, S., Hauer, B., Shi, N., & Kondrak, G. (2023, December). Don't trust ChatGPT when your question is not in English: A study of multilingual abilities and types of LLMs. In *Proceedings of the 2023 conference on empirical methods in natural language processing* (pp. 7915-7927). 2023.emnlp-main.491.pdf

Appendices

A). Ontology Schema

Table 1: Rudimentary Ontology Schema

Predicate (URI/Naam)	Domain (Subject Class)	Range (Object Class / Datatype)	Explanation
appearsIn	Character	Story	Which character appears in the story?
defeats	Character (or Hero)	Character (or Villain)	Who defeated who in the story?
discoversLocation	Character	Location	Which location is being discovered by a character?
executesAction	Character	xsd:string	A character executes an action, for example: walking, drinking
findsObject	Character	Object	A character finds an object
givesObject	Character	Object	A character gives an object to someone
hasDescription	Character/Location/Item/Object	xsd:string	A character, location, item or object has a description, example: is loud, has many (...)
hasEmotion	Character	Emotion	A character has an emotion, example: sad, happy, angry
hasEnding	Story	xsd:string	A story has an ending
hasLocation	Story / Character / Action	Location	The location of the story, for example a town
hasSubGenre	Story	xsd:string	The story has a subgenre, example: sprookje, sage, mop.
hasTime	Story / Action	Temporal	The story or an action has a time, example: yesterday, tomorrow
helps	Character	character (or Villain)	A character helps someone
livesIn	Character	Location	A character lives in a place, example: Groningen
transformsInto	Character	Character (or Villain)	A character transforms into someone
usesObject	Character	Object	A character uses a magical object, example: broom

B). Distribution of work:

Stage 1: Brainstorming, Planning and Task Formulation:
Defining the research problem and research questions, reviewing relevant literature, selecting the methodology, and planning the experiments and project approach.

Jasper: 25%
Marten: 35%
Fleur: 40%

Equally distributed: curation of questions per text & language.

Stage 2: Experiments and Code:
Preparing the data, implementing the solution and knowledge graph, running experiments, debugging the code, and evaluating the results.

Jasper: 80%
Marten: 15%
Fleur: 5%

Stage 3: Report Writing:
Writing the report, documenting the methodology and results, discussing the findings, and formulating conclusions and limitations.

Jasper: 5%
Marten: 45%
Fleur: 50%